Spring Boot – Day 03 & 04 Notes

Auto Configuration, Core Features & Project Setup

Feature 1: Auto Configuration

What is Auto Configuration?

Auto Configuration is a feature of Spring Boot that automatically configures the required components for an application based on the dependencies available in the project. It helps us build the functional part of the application faster because Spring Boot performs most of the configuration work automatically.

> **Analogy:** Auto Configuration is like moving into a fully furnished apartment instead of an empty one — the furniture (configuration) is already set up based on what kind of apartment (application) you booked, so you can start living (coding) right away instead of buying and arranging everything yourself.

Why did this feature come?

In the Spring Framework, we had to configure many things manually before writing the actual functionality.

We had to write:

- XML Configuration
- Java Configuration
- Bean Configuration
- Server Configuration
- Database Configuration

Because of this, development became slow and there were more chances of configuration mistakes.

Spring Boot introduced Auto Configuration to reduce this manual work.

How does Auto Configuration help?

When we add a suitable Starter Dependency, Spring Boot automatically understands what type of application we are creating and configures the required components for us.

For example, if we add Spring Boot Starter Web, Spring Boot automatically prepares the web application by configuring the required web-related components.

Because of this:

- No XML configuration is required.
- No separate Java configuration is required in most cases.

- The required libraries come through Starter Dependencies.
- The embedded server is configured automatically.
- We can focus more on writing the application functionality instead of configuration.

Advantage

- Faster application development.
- Less configuration code.
- Fewer configuration errors.
- We can directly focus on business logic.

2. Starter Dependencies

What is it?

Starter Dependency is a ready-made dependency that contains all required libraries for a particular feature.

Why did this feature come?

Earlier we searched every dependency separately.

Example:

```
Spring Core
Spring Context
Spring JDBC
Tomcat
```

Now one starter brings everything together.

> **Analogy:** A Starter Dependency is like ordering a combo meal at a restaurant instead of ordering the burger, fries, and drink separately — one combo (starter) brings everything that naturally belongs together, already matched and compatible.

Advantage

- No need to search multiple dependencies.
- Saves time.
- Compatible versions come together.

Example

For a web application:

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

This single dependency brings:

- Spring MVC
- Embedded Tomcat
- Jackson
- Validation libraries

3. Embedded Server

What is it?

Spring Boot already provides a server inside the project.

Default server:

```
Tomcat
```

Other supported servers:

- Jetty
- Undertow

Why did this feature come?

Earlier servlet projects required:

- installing Tomcat
- configuring Tomcat
- deploying WAR file manually

Spring Boot removes these steps.

> **Analogy:** An Embedded Server is like buying a car that already comes with an engine installed, instead of buying the car and the engine separately and fitting it yourself — you just turn the key and drive.

Advantage

- No manual server installation.
- Run application directly.
- Faster testing.

Example

Run the application.

Output:

```
Tomcat started on port 8080
```

The server starts automatically.

4. Spring Initializr

What is it?

Spring Initializr is an online tool used to create a Spring Boot project.

Why did this feature come?

Earlier we created Maven projects manually and added every dependency ourselves.

Now Spring Initializr creates the project structure automatically.

> **Analogy:** Spring Initializr is like a cake-mix box — instead of measuring and gathering flour, sugar, and baking powder separately, you pick the flavor and quantity you want, and it hands you a ready base structure to build on.

Advantage

- Ready project structure
- Faster project creation
- Easy dependency selection

5. DevTools

What is it?

DevTools automatically restarts the application after code changes.

Why did this feature come?

Earlier after every code change we had to:

- stop server
- run server again

DevTools removes this repeated work.

> **Analogy:** DevTools is like a phone that automatically refreshes an app the moment you update its settings, instead of you having to force-close and reopen the app every single time.

Advantage

- Automatic restart
- Faster development
- Better productivity

Dependency

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>
```

Example

- Run application.
- Change one Java file.
- Save.
- Application restarts automatically.

No need to restart manually.

What is CLI?

CLI (Command Line Interface) is a feature of Spring Boot that allows us to create and run a Spring Boot application by using commands instead of depending completely on an IDE like Eclipse or IntelliJ.

It is mainly useful for quickly creating, running, and testing a Spring Boot application from the command line.

Why did this feature come?

Earlier, we mostly depended on an IDE to create and run our projects. If we wanted to run the application outside the IDE, we had to compile and execute it manually.

Spring Boot CLI was introduced to make application execution faster from the command line.

> **Analogy:** CLI is like using keyboard shortcuts instead of clicking through menus with a mouse — it does the same job, just faster, once you're comfortable typing the commands directly.

How does CLI help?

- Run a Spring Boot application using commands.
- Test the application quickly.

- Useful for rapid development.
- No need to depend on the IDE for every execution.

Advantage

- Faster execution.
- Easy testing.
- Less dependency on the IDE.
- Quick development during practice.

What is Spring Boot Actuator?

Spring Boot Actuator is a feature that helps us monitor and check the health and working status of a running application. It provides ready-made endpoints that tell us whether the application is running properly or if there is any problem.

Think of it as a watchdog for the application—it keeps an eye on the application's condition while it is running.

Why did this feature come?

Earlier, if we wanted to know whether the application was working correctly, we had to check the server logs or create our own monitoring code.

Spring Boot introduced Actuator so that we can quickly check the application's health and other information through built-in endpoints.

> **Analogy:** Actuator is like the dashboard in a car — instead of opening the hood every time to check the engine, fuel, and temperature, the dashboard gives you ready-made gauges that show the car's health at a glance.

How does Actuator help?

- Checks whether the application is Up or Down.
- Provides ready-made monitoring endpoints.
- Helps developers verify that the application is running correctly.
- Makes monitoring easier without writing extra code.

Advantage

- Quick health checking.
- Easy application monitoring.
- No need to write custom monitoring code.
- Built-in endpoints save development time.

Project Structure

```
DemoProject

src/main/java
    DemoApplication.java

src/main/resources
    application.properties

pom.xml
```

Important files:

File	Purpose
`DemoApplication.java`	Starts application
`application.properties`	Configuration
`pom.xml`	Dependencies

Creating Spring Boot Project using Spring Initializr

Follow these steps exactly.

Step 1

Open:

```
https://start.spring.io
```

Step 2

Choose

Option	Value
Project	Maven
Language	Java
Spring Boot	Latest Stable
Packaging	Jar
Java	17

Step 3

Enter project details.

Example:

Field	Value
Group	com.demo
Artifact	FirstSpringBoot
Name	FirstSpringBoot
Package	com.demo

Step 4

Click Add Dependencies.

Add:

- Spring Boot Starter Web
- Spring Boot DevTools

Step 5

Click

```
Generate
```

A ZIP file downloads.

Extract the ZIP File

After downloading:

- Open Downloads.
- Right click ZIP file.
- Click Extract All.
- Choose location.
- Click Extract.

Now the project folder is ready.

Example:

```
FirstSpringBoot
```

Inside it:

```
src
pom.xml
mvnw
mvnw.cmd
```

Import Spring Boot Project into Eclipse

After extracting the ZIP:

Step 1

Open Eclipse.

Step 2

Click

```
File
↓
Import
```

Step 3

Select

```
Maven
↓
Existing Maven Projects
```

Click

```
Next
```

Step 4

Click

```
Browse
```

Select the extracted project folder.

Example:

```
FirstSpringBoot
```

Click

```
Select Folder
```

Step 5

Eclipse automatically detects

```
pom.xml
```

Click

```
Finish
```

Step 6

Wait for Maven.

It downloads all required dependencies.

After completion the project appears in Project Explorer.

Running the Application

Open

```
DemoApplication.java
```

Right Click

```
Right Click
     ↓
  Run As
     ↓
Spring Boot App
```

or

```
  Run As
     ↓
Java Application
```

Maven Project vs Spring Boot Project

Maven Project	Spring Boot Project
Spring Core manually	Starter Dependency
Spring Context manually	Included automatically
JDBC manually	Starter Data JPA
Tomcat manually	Embedded Tomcat
More configuration	Auto Configuration